CAMPUS FOOD CANTEEN ORDERING SYSTEM

Java Object-Oriented Programming Laboratory Exercise

Student Name: _____

Section: _____

Date: _____

Exercise: Campus Food Canteen Ordering System

Project Description

This project is a console-based ordering system for a campus food canteen. Customers can order up to three products, including beverages, snacks or pastries, and rice meals. Student customers receive a 10% discount, and customers earn one loyalty point for every 50 pesos paid.

OOP Concepts Used

Classes and Objects: The program uses seven classes: MenuItem, Beverage, Pastry, Meal, Customer, Order, and CafeApp. CafeApp creates the customer, product, and order objects.

Object Interaction: Order stores a Customer and up to three MenuItem objects. It calls each product's methods to calculate prices and prints the customer's receipt.

Abstraction: MenuItem is an abstract class with the abstract computePrice() method. Each product subclass provides its own price calculation.

Constructors: Default and parameterized constructors initialize products. Customer also demonstrates constructor overloading with three constructor forms.

Encapsulation: Product and customer data are private or protected. Getters and setters control access, and setters reject invalid values such as negative prices or extra sides.

Method Overloading: Customer has addPoints(int) and addPoints(double). Order has computeTotal() and computeTotal(double) to support voucher discounts.

Method Overriding / Polymorphism: Beverage, Pastry, and Meal override computePrice() and getDescription(). Order calls computePrice() through MenuItem references.

Conditional Statements: if and if-else statements validate values, calculate size and warming fees, enforce the three-item limit, and apply student discounts.

Challenges and Solutions

One challenge was making different product types work in one order while calculating different prices. This was solved by using the abstract MenuItem parent class and overriding computePrice() in each subclass. Another challenge was preventing invalid input. Validation in setters keeps negative prices and extra-side counts from being stored.

Program Output

The screenshot below was generated from the program after compiling and running CafeApp. The project was tested successfully with javac and java CafeApp.

CanteenApp — actual Java console output

```
CAMPUS FOOD CANTEEN ORDERING SYSTEM
Error: base price cannot be negative.
Order is full. Cannot add Cookie.

============================================
ORDER #1001 - Ana Reyes (Student)
--------------------------------------------
Iced Tea [B-01] Size: L            PHP  120.00
Cheese Bread [P-01] (Warmed)       PHP   55.00
Chicken Adobo [M-01] Upsized, 1 side(s) PHP  142.00
Subtotal:                         PHP 317.00
Student discount (10%):          -PHP 31.70
TOTAL:                            PHP 285.30
Loyalty points: 5
============================================

============================================
ORDER #1002 - Ben Cruz
--------------------------------------------
Hot Chocolate [B-02] Size: M       PHP   95.00
Beef Tapa [M-02] Regular, 0 side(s) PHP  130.00
Subtotal:                         PHP 225.00
Voucher:                          -PHP 20.00
TOTAL:                            PHP 205.00
Loyalty points: 4
============================================

Additional default-constructor examples:
Banana Cue [P-03] - PHP 20.0
Pork Sisig Rice [M-03] Regular, 0 side(s) - PHP 105.0
Lemon Juice [B-03] Size: M - PHP 95.0
Guest created: Guest
```